```python
def generate_image(prompt, filename=None):
    if not filename:
        filename = sanitize_filename(prompt)

    image = pipe(prompt).images[0]
    path = f"static/panels/{filename}"
    os.makedirs(os.path.dirname(path), exist_ok=True)
    image.save(path)
    return path
```

Organize Comic Panels into Layout Structure

Functions: build_comic_layout()

File: layout_builder.py

- This function organizes the generated images and full comic story into a structured layout.
- It matches each image with its corresponding panel story.
- The output is a list of dictionaries containing the panel number, image path, and text.

```python
def build_comic_layout(image_paths, full_story, outline):
    # Split the full story into individual panel segments
    story_panels = full_story.split("**Panel")
    story_panels = [f"**Panel{panel}" for panel in story_panels if panel.strip()]

    # Build layout panel by panel
    layout = []
    for idx, (image, text, panel_info) in enumerate(zip(image_paths, story_panels, outline), start=1):
        layout.append({
            "panel": idx,
            "title": panel_info.get("title", f"Panel {idx}"),
            "image_path": image,
            "text": "\n".join(text.strip().splitlines()[1:]).strip(),  # remove panel title
            "scene_description": panel_info.get("scene_description", "")
        })

    return layout
```

Export Comic Panel and Story into PDF

Function: save_pdf()
File: exporters.py

- This function compiles the full comic into a multi-page **PDF file** using the FPDF library.
- Each panel's image and narration are placed neatly on separate pages.
- The final PDF file is saved into the static/exports folder, and the path is returned.

```
def save_pdf(layout):
    pdf = PDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    # Add Unicode font
    pdf.add_font('DejaVu', '', FONT_PATH, uni=True)
    pdf.set_font('DejaVu', '', 12)

    for panel in layout:
        image_path = panel['image_path']
        story_text = panel['text']

        pdf.add_page()

        # Panel title (no bold)
        pdf.set_font('DejaVu', '', 14)
        pdf.cell(0, 10, f"Panel {panel['panel']}", ln=True, align="C")
        pdf.set_font('DejaVu', '', 12)

        # Image placement
        y_image = 30
        image_height = 100
        spacing_after_image = 15

        if os.path.exists(image_path):
            pdf.image(image_path, x=10, y=y_image, w=pdf.w - 20, h=image_height)
        else:
            pdf.set_y(y_image)
            pdf.multi_cell(0, 10, f"Image missing: {image_path}")

        # Text placement below image
        pdf.set_y(y_image + image_height + spacing_after_image)
        story_lines = story_text.strip().splitlines()
        # Remove title line like "**Panel 1: Title**" if present
        if story_lines and story_lines[0].strip().lower().startswith("**panel"):
            story_lines = story_lines[1:]
        cleaned_text = "\n".join(story_lines).strip()
        pdf.multi_cell(0, 10, cleaned_text)

    # Save with timestamp
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    pdf_path = os.path.join(EXPORT_FOLDER, filename)
    pdf.output(pdf_path)

    return pdf_path
```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in **routes.py**.
Each route is linked to the corresponding backend function that processes the comic generation workflow developed in Activity 2.1.

Process User Input

- **HTML Form**: Created in index.html
- **Users enter**: story prompt, main character name, setting, tone, and art style
- **FastAPI captures inputs** using Form(...) parameters from the request
- **Pydantic schema** (PromptRequest) is used to validate JSON input for API-based comic generation

Integrate Gemini API Calls

Each route handler connects directly to the respective AI-powered function modules:

- **/generate** →
  - Calls generate_outline() to create panel outlines from user prompt
  - Calls generate_story() to expand into narration and dialogue
  - Calls generate_image() for each panel to create comic-style images
  - Calls build_comic_layout() to organize the final layout
  - Calls save_pdf() to compile and export the complete comic into a PDF
- **/generate-comic/json** →
  - Similar flow as /generate, but triggered via a raw JSON API request
  - Returns JSON response with layout data and PDF path for API clients
- **/test-image** →
  - Calls generate_image() directly with a custom prompt to test image generation functionality separately

All AI-generated responses (text and images) are processed, cleaned, and injected into the Jinja2 frontend templates (comic_preview.html, export_success.html) for a seamless user experience.

Milestone 3. routes.py Development